DATA 266 - Homework 8 Findings

Mini Diffusion Model on MNIST: from DDPM to DDIM Sampling

Personal Parameters

Parameter	Value
SID4	9486
SEED	9486
SLICE	486
HP_ID	0
CLS_A	6
CLS_B	0

These are calculated with the same Step 0 formulas as Assignment 1. SEED 9486 is used for Python, NumPy, and PyTorch, the DataLoader shuffling, the timesteps and noise drawn during training, the fixed noise in the forward-diffusion figure, the starting noise x_T, and the noise added during DDPM sampling. SLICE and CLS_A pick the image for the forward-diffusion figure: the first MNIST test image of digit 6 at index 486 or later, which is test image 499. The assignment fixes T, the beta schedule, the batch size, and the number of epochs, so there is no hyperparameter sweep, and HP_ID and CLS_B are only reported.

Introduction

The goal of this homework was to build a small diffusion model in the style of DDPM, show how noise is added to MNIST images and how a CNN learns to reverse it, and then change only the sampling procedure to get DDIM. I started from the professor's Demo_8_Diffusion.ipynb and kept its schedule code, q_sample, recover_x0_from_noise, TimeEmbedding, and denorm. The demo stops after a single forward pass of an untrained 3-layer denoiser, so I added a larger noise-prediction CNN, the training loop, DDPM sampling, DDIM sampling, and the comparison. Every number in this report comes from the executed HW8_Diffusion_DDPM_DDIM.ipynb notebook, run top to bottom on an Apple M1 (PyTorch MPS backend), from 2026-10-27T19:18:48Z to 2026-10-27T20:56:22Z UTC. I ran the full notebook twice and got exactly the same training losses, test MSE, and samples both times; only the timings differed.

What's in this submission

Part	What was completed
Setup	Personal parameters from SID4, seeding and deterministic algorithms, automatic device choice (CUDA, then MPS, then CPU), and a smoke-test switch (HW8_SMOKE_TEST=1) kept separate from the full settings.
A. Forward diffusion	Linear beta schedule (T = 100, 0.0001 to 0.02), the closed-form q_sample, a labeled figure of one test image at t = 0, 10, 30, 60, 100, and three numerical checks of the formula.

B. Mini DDPM	A 15-layer conv encoder-decoder with sinusoidal timestep conditioning in every block, trained for 50 epochs with MSE(ε, $\varepsilon_\theta(x_t, t)$), with per-batch and per-epoch loss curves and a held-out test MSE.
C. DDPM sampling	The full 100-step stochastic reverse process from Gaussian noise, a sample grid, and the reverse process at selected timesteps.
D. DDIM sampling	Deterministic DDIM ($\eta = 0$) with 50 and 20 evenly spaced steps from the same model and the same starting noise, a side-by-side comparison with DDPM, timings, stochasticity checks, and a diagnostic of the sample quality.

Setup

Item	Value
Data	MNIST from torchvision, 60,000 training and 10,000 test images, ToTensor + Normalize(0.5, 0.5) so pixels are in [-1, 1] (same as the demo). The transform is applied once to the whole tensor, and the notebook checks it matches the per-image transform exactly.
Schedule	$T = 100$, β_t linear from 0.0001 to 0.02, $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod \alpha_i$. $\bar{\alpha}_{100} = 0.3636$.
Model	MiniDenoiser: U-Net-style encoder-decoder, 32-64-128 channels, 15 conv layers, GroupNorm + ReLU, 710,593 parameters. Input [B, 1, 28, 28] and t, output [B, 1, 28, 28].
Training	Adam, lr 0.001 with cosine decay to 0, batch 128 (469 updates per epoch), 50 epochs, 97.0 minutes on MPS.
Sampling	64 images per sampler, all from the same $x_T \sim N(0, I)$. DDPM: 100 steps, $\sigma_t^2 = \beta_t$. DDIM: 50 and 20 steps, $\eta = 0.0$.
Software	torch 2.14.0, torchvision 0.29.0, numpy 2.3.5, torch.use_deterministic_algorithms(True, warn_only=True). No warnings were printed in the reported run.

Part A: Forward Diffusion

The forward process adds a little Gaussian noise at every step, $q(x_t \mid x_{t-1}) = N(\sqrt{\alpha_t}\, x_{t-1}, \beta_t I)$. Since a sum of Gaussians is Gaussian, the steps collapse into one closed-form jump from the clean image to any timestep:

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{(1 - \bar{\alpha}_t)}\, \varepsilon, \varepsilon \sim N(0, I)$$

This is what makes training cheap: any x_t can be built in one line without running t noising steps. β_t is the variance of the noise added at step t. It starts tiny, so the first steps barely change the image, and grows so that later steps remove more of the signal. $\bar{\alpha}_t$ is how much of the original image is left after t steps.

t	β_t	$\bar{\alpha}_t$	$\sqrt{\bar{\alpha}_t}$ (image)	$\sqrt{(1 - \bar{\alpha}_t)}$ (noise)
10	0.00191	0.9900	0.995	0.100
30	0.00593	0.9134	0.956	0.294
60	0.01196	0.6954	0.834	0.552
100	0.02000	0.3636	0.603	0.798

The most important number here is $\bar{\alpha}_{100} = 0.3636$: even at the last step, x_{100} still contains $0.60 \cdot x_0$. The $\beta_T = 0.02$ endpoint comes from the original DDPM paper, which used T = 1000, so its $\bar{\alpha}_T$ is almost 0. With only 100 steps the forward process never reaches pure noise. Figure 2 shows this directly: the 6 is still faintly visible at t = 100. This turns out to explain most of what the samples look like (see the diagnostic below).

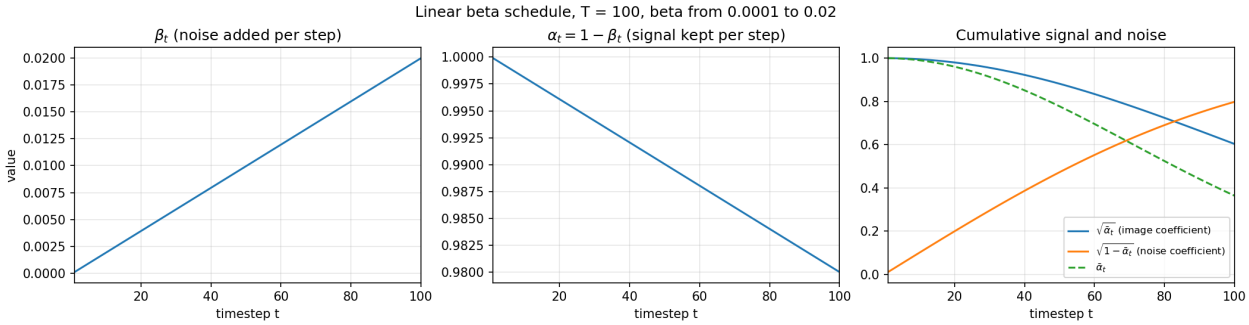


Figure 1. The linear beta schedule: β_t, α_t, and the cumulative image and noise coefficients.

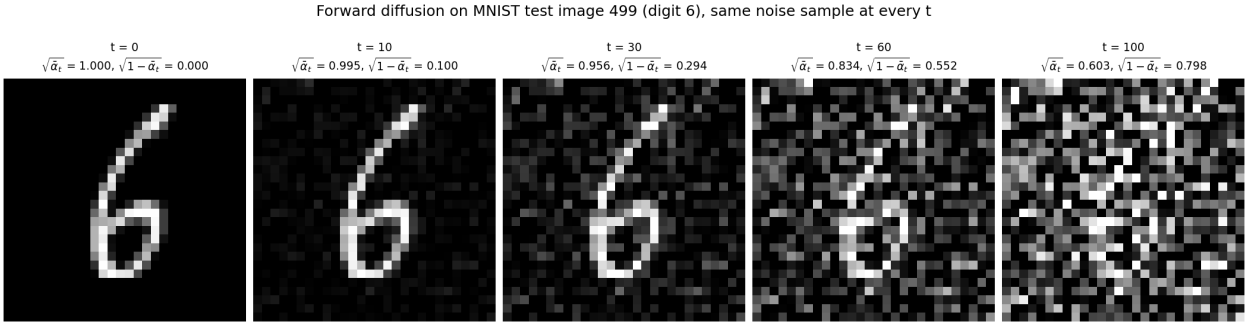


Figure 2. Forward diffusion of test image 499 (digit 6) at t = 0, 10, 30, 60, 100, using the same noise sample ε at every t, so only the mix changes.

Three checks passed in the reported run: the output shape equals the input shape; solving the formula for x_0 with the true ε recovers the clean image (maximum error 1.19e-07 at t = 60); and over 1,000 test images at t = 100, the mean and standard deviation of x_t (−0.4571 and 0.8741) match the values the formula predicts (−0.4555 and 0.8752).

Part B: Training the Mini DDPM

Noise-prediction model

The network $\varepsilon_\theta(x_t, t)$ takes a noisy image and its timestep and outputs a tensor of the same shape as the image, its guess for the noise that was added. It is a small U-Net-style encoder-decoder: two ConvBlocks at 28×28 and 14×14, strided convolutions to downsample to 7×7, a 128-channel middle block, and transposed convolutions back up to 28×28 with skip connections from the encoder. Each ConvBlock is Conv 3×3, GroupNorm, ReLU, then the time feature is added, then Conv 3×3, GroupNorm, ReLU. A final 1×1 convolution with no activation gives the noise, which can be any real number. The notebook checks that the output shape equals the input shape and that there are at least four conv layers (there are 15).

Timestep conditioning. One network denoises at all 100 noise levels, so it must be told t. The integer t first goes through the demo's sinusoidal embedding (sin and cos of t at 64 frequencies, like transformer positional encodings), then a small MLP (Linear, ReLU, Linear) to a 128-d time feature. Every ConvBlock has its own Linear(128, channels) that turns this into one value per channel, which is added to the feature map after the first convolution and broadcast over all pixels. So t shifts each channel up or down, letting the same filters behave differently at different noise levels. The demo only added the time embedding once, before its last layer. Before training, changing only t from 1 to 100 already changed the output (mean difference 0.0416), so the conditioning path is connected.

Training

Each step takes a batch of clean images, draws a random t for each image and $\varepsilon \sim N(0, I)$, builds x_t with q_sample, and minimizes $L = \text{MSE}(\varepsilon, \varepsilon_\theta(x_t, t))$. Because t is random, every batch trains the model on many noise levels at once.

Epoch	1	5	10	25	50
Mean training MSE	0.12903	0.06317	0.05881	0.05445	0.05244

The loss fell quickly in the first epoch (the very first batch was 1.290, the epoch-1 mean 0.12903) and then kept going down slowly to 0.05244 at epoch 50. The lowest epoch mean was 0.05219 at epoch 46, and the last five epochs stayed between 0.05219 and 0.05255, so training had mostly converged. On the 10,000 test images, with fixed seeded timesteps and noise, the MSE went from 1.28678 before training to 0.05245 after, almost the same as the final training loss, so there is no sign of overfitting. The per-batch loss stays noisy because each batch has a different random mix of timesteps.

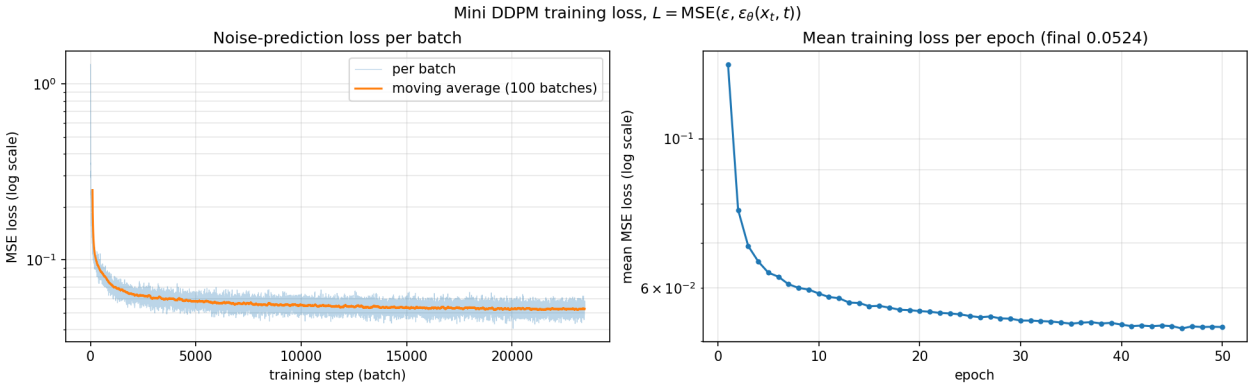


Figure 3. Training loss per batch with a 100-batch moving average (left) and mean loss per epoch (right), both on a log scale.

Part C: DDPM Sampling

Sampling runs the chain backwards from $x_T \sim N(0, I)$. For t = 100, ..., 1:

$$x_{t-1} = (1 / \sqrt{\alpha_t}) \cdot (x_t - (\beta_t / \sqrt{(1 - \bar{\alpha}_t)}) \cdot \varepsilon_\theta(x_t, t)) + \sigma_t z, \; z \sim N(0, I)$$

The first part is the model's estimate of the mean of x_{t-1}: it removes the part of the predicted noise that belongs to step t and undoes the $\sqrt{\alpha_t}$ shrinking. Then fresh noise is added with $\sigma_t^2 = \beta_t$, except at the last step, so the output is the clean estimate. Each step only moves from t to t − 1, so one image costs 100 network calls. Generating 64 images took 3.56 s on MPS.



Figure 4. 64 DDPM samples (100 steps) from the seeded starting noise.

The DDPM samples look like MNIST in style: thick, smooth white strokes on a clean black background with almost no speckle. Most of them are not clearly readable digits, though. A few look like a 9, 7, 5, 3, or 2, but many are loops and strokes that look digit-like without being any particular digit. They also have more ink than real digits (ink fraction 0.218 vs 0.124 for real MNIST), so the strokes are thicker and cover more of the image.

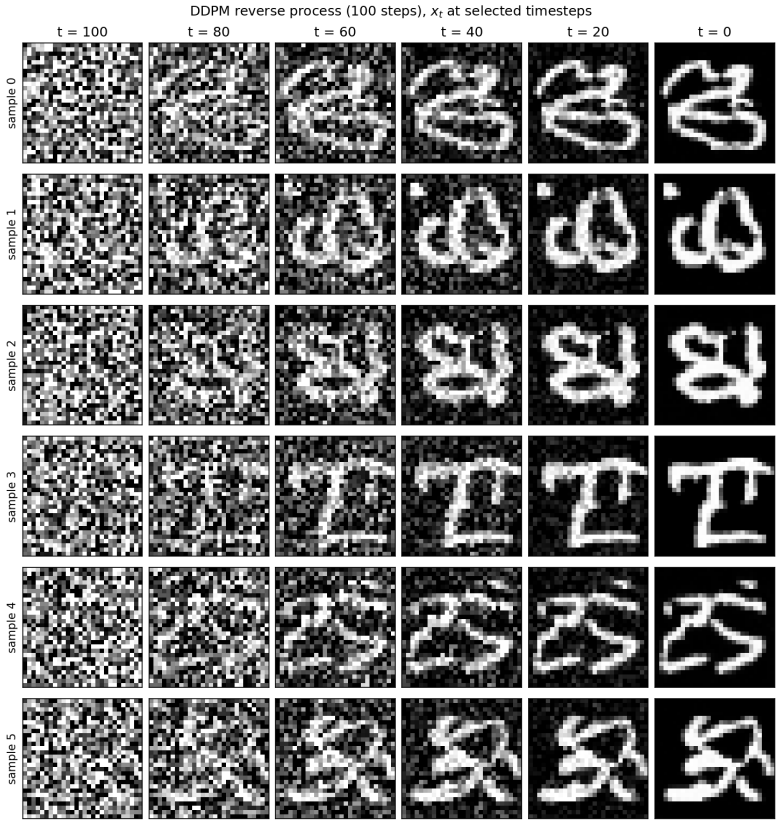


Figure 5. DDPM reverse process for the first six samples, x_t at t = 100, 80, 60, 40, 20, 0.

In the reverse process, x_{100} and x_{80} still look like random noise. The rough layout of the strokes appears around t = 60, and from t = 40 to t = 0 the model mostly cleans up the background and sharpens the strokes that are already there. So the big decisions about the shape are made early, at the high-noise steps, and the low-noise steps only refine it.

Part D: DDIM Sampling

DDIM (Song et al., 2021) reuses the same trained model; only the sampling loop changes. DDPM training only ever uses the closed-form marginals $q(x_t \mid x_0)$, and DDIM defines a different (non-Markovian) reverse process with the same marginals, so ε_θ is still valid but each step can jump between any two timesteps $t \to s$. Each step first predicts the clean image and then re-noises it to the earlier level:

$$\hat{x}_0 = (x_t - \sqrt{(1 - \bar{\alpha}_t)}\, \varepsilon_\theta(x_t, t)) / \sqrt{\bar{\alpha}_t}$$
$$x_s = \sqrt{\bar{\alpha}_s}\, \hat{x}_0 + \sqrt{(1 - \bar{\alpha}_s - \sigma_t^2)}\, \varepsilon_\theta(x_t, t) + \sigma_t z$$
$$\sigma_t = \eta \cdot \sqrt{((1 - \bar{\alpha}_s) / (1 - \bar{\alpha}_t))} \cdot \sqrt{(1 - \bar{\alpha}_t / \bar{\alpha}_s)}$$

Role of η. With η = 0, which I used, $\sigma_t = 0$: no random noise is added, and the output is a deterministic function of x_T. With η = 1 it adds fresh noise like DDPM. **Reduced schedule.** I use evenly spaced timesteps from 100 down to 0: 100, 98, ..., 2, 0 for 50 steps and 100, 95, ..., 5, 0 for 20 steps. The notebook checks that both schedules start at T, end at 0, and strictly decrease. The last jump goes to s = 0 where $\bar{\alpha}_0 = 1$, so the output is $\hat{x}_0$. **Why fewer steps work.** DDPM's update depends on the single-step β_t and adds new noise every step, so it has to visit every t. The DDIM update only needs $\bar{\alpha}$ at the two ends, which is known in closed form for any pair, and with η = 0 the path from x_T to x_0 is

smooth, so bigger steps along it are still a good approximation.



Figure 6. DDPM (100 steps), DDIM (50 steps, η = 0), and DDIM (20 steps, η = 0), all starting from the same 64 noise images.

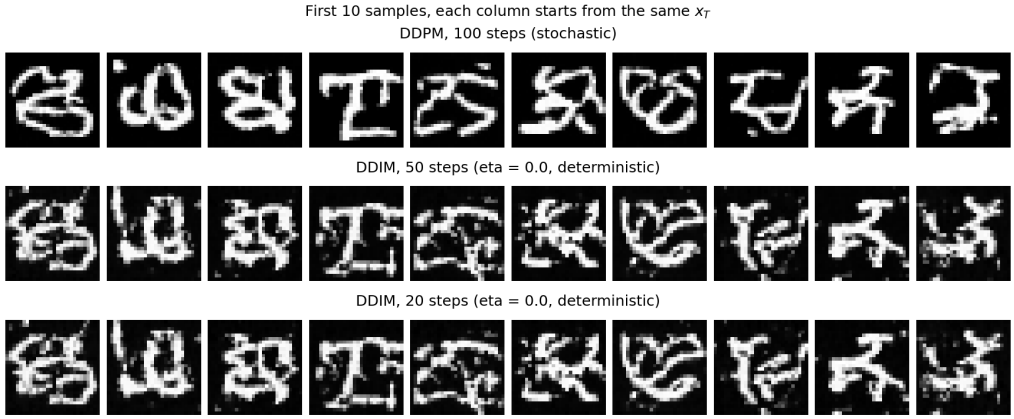


Figure 7. The first 10 samples of each method; each column starts from the same x_T.

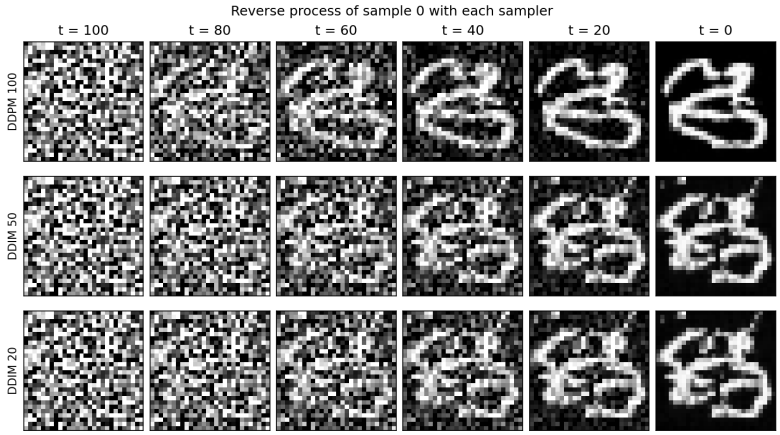


Figure 8. Reverse path of sample 0 under each sampler at t = 100, 80, 60, 40, 20, 0.

Comparison numbers

FID would need a pretrained Inception network, so I used simple statistics instead: mean pixel value, ink fraction (pixels above 0.5 on the [0, 1] scale), gray fraction (pixels between 0.1 and 0.9, which goes

up with speckle or blur, since real digits are mostly pure black and white), and the mean absolute difference from the DDPM image that started from the same noise.

Method	Steps	Time, 64 images	Mean pixel	Ink fraction	Gray fraction	Diff vs DDPM
Real MNIST (1,000 test images)	-	-	0.1223	0.1239	0.0850	-
DDPM	100	3.56 s	0.2167	0.2181	0.1596	0.0000
DDIM, η = 0	50	1.78 s	0.2604	0.2463	0.2493	0.1407
DDIM, η = 0	20	0.70 s	0.2595	0.2445	0.2531	0.1402

Stochasticity: running DDIM 20 a second time from the same x_T gave a maximum pixel difference of exactly 0.0. Running DDPM a second time from the same x_T with a different noise seed changed the images by 0.1450 per pixel on average (on the [0, 1] scale), and the two runs give clearly different digits (figures/ddpm_two_runs.png). DDIM 50 and DDIM 20 differ from each other by only 0.0046 per pixel.

Diagnostic: why the samples look the way they do

The DDIM samples were worse than DDPM even though DDIM uses the same model, so I added a check to tell a training problem apart from a sampling problem. I took 10 real test images, made a real x_{100} from each with the forward process (which still has $0.60 \cdot x_0$ in it), and ran both samplers from there instead of from pure noise. I also ran DDIM 20 with η = 1 from the same pure noise x_T used in the main comparison.

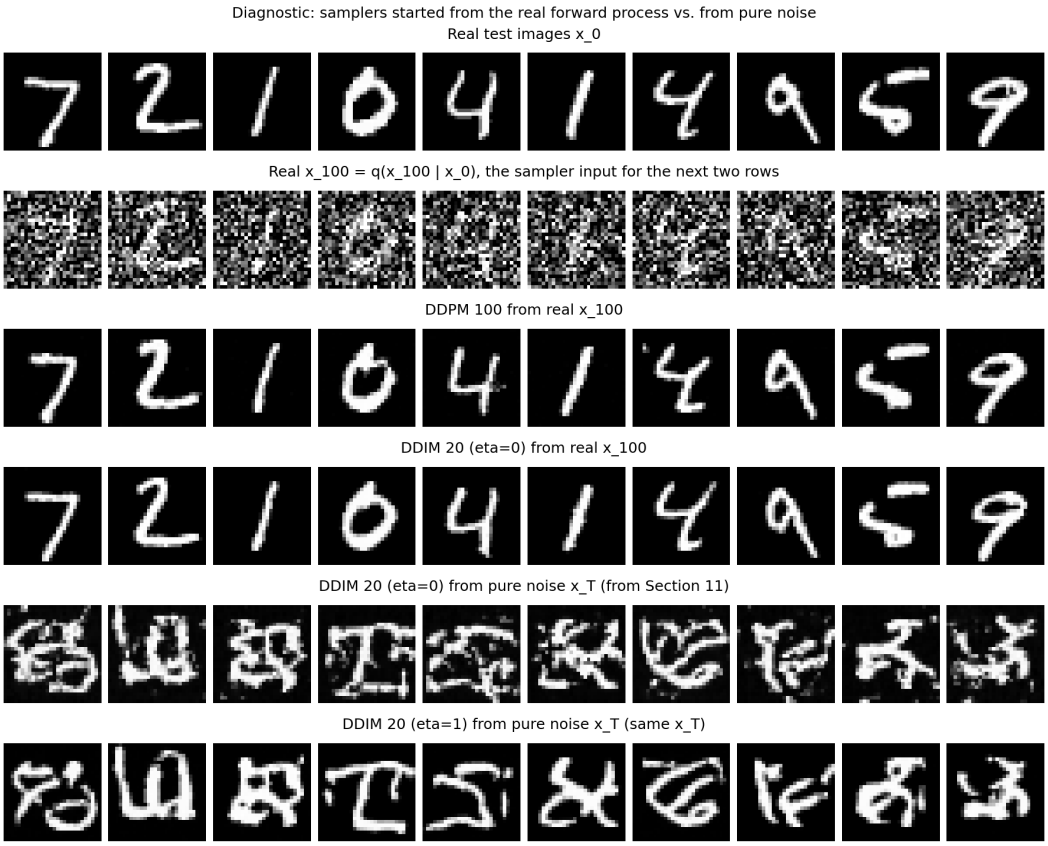


Figure 9. Diagnostic. Rows: real test images; their x_{100} from the real forward process; DDPM and DDIM 20 started from that x_{100}; DDIM 20 ($\eta = 0$) from pure noise; DDIM 20 ($\eta = 1$) from the same pure noise.

Starting from a real x_{100}, both samplers gave back clean, readable digits that match the originals: the mean absolute error to x_0 was 0.0399 for DDPM and 0.0313 for DDIM 20. So the trained model and both samplers work correctly. The problem is the starting point: pure N(0, I) noise does not look like a real x_{100} under this schedule, because $\bar{\alpha}_{100} = 0.364$ instead of about 0. DDPM adds fresh noise at every step, which seems to let it correct this mismatch on the way down. DDIM with $\eta = 0$ is fully deterministic, so the bad start is carried all the way to the output, and that shows up as speckle and messy strokes. With $\eta = 1$ from the same pure noise, DDIM 20 gave smooth samples again (gray fraction 0.164, close to DDPM's 0.160, compared with 0.253 at $\eta = 0$), which supports this explanation.

Comparison

Speed. Runtime follows the number of network calls: DDPM 100 steps took 3.56 s for 64 images, DDIM 50 took 1.78 s (2.0x faster), and DDIM 20 took 0.70 s (5.1x faster). **Fewer steps.** DDIM 50 and DDIM 20 are almost identical (mean difference 0.0046), so in this model dropping from 50 to 20 steps cost nothing visible. **Stochasticity.** DDIM with $\eta = 0$ is deterministic (repeat difference exactly 0.0), while DDPM gives a different image each time from the same x_T (mean difference 0.1450). **Quality.** DDPM gave the cleaner samples in this setup. Its gray fraction was 0.160 vs about 0.25 for both DDIM runs (real MNIST is 0.085), and its strokes are smooth while DDIM's are speckled. DDIM samples share the rough layout of the DDPM sample from the same x_T but differ in detail (mean difference about 0.14). The reverse paths in Figure 8 show that DDIM keeps more noise until t = 40, while DDPM already has clear structure at t = 60. Based on the diagnostic, the quality gap comes from the short

linear schedule combined with $\eta = 0$, not from DDIM itself.

What Could Improve the Model

The biggest improvement would be a noise schedule that actually reaches pure noise at T, so that sampling starts from the same distribution the model was trained on. Options are a larger final β, more steps (the original DDPM used $T = 1000$ with this β range), or a cosine schedule. With that fixed, DDIM with $\eta = 0$ should match DDPM much more closely. Other things that would likely help: a bigger U-Net with attention at 7×7 and more channels, an EMA copy of the weights for sampling, training longer, and class conditioning (or classifier-free guidance) to ask for a specific digit. For DDIM specifically, a small η (for example 0.2 to 0.5) is a cheap way to get some of DDPM's correction while keeping most of the speed.

Limitations

This is one run with one seed (9486) on an Apple M1 through MPS, so I cannot say how much the results vary across seeds. Sample quality is judged only by looking at the grids and by simple pixel statistics (ink and gray fraction). I did not compute FID or use a digit classifier, so statements about quality are qualitative. The timings are from a single run on a shared laptop GPU and were a little different in my two full runs (DDPM 4.36 s vs 3.56 s), although the ranking was the same. The assigned schedule leaves $\bar{\alpha}_T = 0.364$, which limits sample quality from pure noise. I kept it because it was part of the assignment. The model is unconditional, so it cannot be asked for a particular digit.

Final findings

The full pipeline worked: forward diffusion with the closed-form q_sample, a 15-layer time-conditioned noise predictor trained to a test MSE of 0.05245, 100-step DDPM sampling, and DDIM sampling with 50 and 20 steps from the same model. DDIM was 2.0x and 5.1x faster than DDPM, fully deterministic with $\eta = 0$, and going from 50 to 20 steps barely changed the output. DDPM gave cleaner, smoother samples in this setup, but most were digit-like scribbles rather than clear digits. The diagnostic showed that the model and samplers can recover real digits when they start from a real x_{100}, so the weaker samples come from the $T = 100$ linear schedule not reaching pure noise. DDPM's fresh noise partly hides this mismatch, while deterministic DDIM does not.